当代人工智能实验二：搜索算法
实验报告

姓名：郭婷月　　学号：10244507404

代码仓库链接：https://github.com/Caraice/Contemporary_AI_2

1 实验目标与实施过程

本次实验围绕 A*、Minimax 和 Alpha-beta 剪枝完成四道题。我先以题目给定样例建立正确性基线，再比较启发函数、搜索深度和访问顺序，最后加入同分处理与节点排序两项改进。四题都得到了预期结果；后续改进我没有改变最优答案，但减少了部分场景中的搜索量。

2 实验设置与环境

实验环境为 Windows 11、Python 3.14.6，算法只使用标准库，绘图使用 Matplotlib。样例计时先预热一次，再运行 31 次取中位数；扩展地图重复 7 次。题四先计算距离矩阵，博弈计时不包含这部分，矩阵预计算中位耗时为 1.2962 ms。

A* 的 expanded 统计非过期堆项的弹出次数，包含起点和终点；generated 统计入堆次数，包含重复改进项。博弈的 visited 统计递归调用数，包含根和叶。截断次数只统计确实跳过剩余孩子的情况。小样例耗时很短，因此分析以节点数为主，时间作为补充。

3 题一：迷宫寻路

我用坐标作为状态，g 记录起点到当前格的最小已知代价，h 采用到终点的曼哈顿距离，优先队列按 f 从小到大弹出节点。每步代价为 1，四方向移动，障碍 X 不可进入。发现更小的 g 时更新父指针；旧堆项弹出时跳过，找到终点后沿父指针恢复路径。无解时题一返回空列表，题二返回 −1。同 f 的 FIFO 指先入队者先访问；题四自然顺序为 T1、T2、T3、T4。

$$f(n) = g(n) + h(n)$$

表 1 题一启发函数与同分规则对照

配置	最短步数	expanded	generated	中位耗时 ms
h 为 0　同分 FIFO	7	16	17	0.0273
曼哈顿　同分 FIFO	7	14	16	0.0303
曼哈顿　同分优先较大 g	7	8	11	0.0218

得到的路径为 (0,0)→(0,1)→(0,2)→(1,2)→(1,3)→(1,4)→(2,4)→(3,4)，共 7 步。起点 g=0、h=7，沿这条路径每前进一步，g 加 1、h 减 1，终点为 g=7、h=0。普通曼哈顿将弹出数从 16 降到 14；继续在相同 f 下优先较大 g，降到 8。这个规则会更早推进到终点附近，但是否有效还要看地图结构，后面用额外场景检验。

4 题二：加权地形与启发函数修正

题二的 g 累加进入下一个格子的地形权值，起点不收费，E 的进入代价为 0。这里不能直接照搬题一的曼哈顿距离：在终点旁边的格子，剩余真实代价为 0，曼哈顿却为 1，已经高估。即使样例碰巧输出正确，也不能据此保证算法对其他输入仍然最优。

因此，我保留曼哈顿距离提供的步数信息，再扣除免费进入终点的最后一步。设 d 为到终点的曼哈顿距离，其他可通行格的最小进入代价为 m，使用下面的下界；本题样例 m 为 1。

$$h(n) = m \cdot max(d(n) - 1, 0)$$

任何到终点的路径至少需要 d 步，最后一步免费，其余至少 d−1 步各付出 m，所以该估计可采纳。对非终点相邻边，h 的下降最多为 m，不超过进入代价；对进入终点的边，两端 h 都为 0，也满足一致性。若存在其他零代价格，m 为 0，算法安全退化为 Dijkstra。

表 2 题二最优路径代价累计

步骤	所在位置	进入代价	累计代价
0	(0,0) 起点	0	0
1 至 3	(1,0) → (2,0) → (3,0)	每步 1	1 → 2 → 3
4 至 7	(3,1) → (3,2) → (4,2) → (4,3)	每步 1	4 → 5 → 6 → 7
8	(4,4) 终点	0	7

这条路线走了 8 步，总代价却是 7，说明步数和权值之和需要分开看。h 为 0 时弹出 15 个节点，修正启发函数后为 11 个；在此基础上优先较大 g 仍为 11 个，没有进一步减少。对应中位耗时为 0.0407、0.0323、0.0329 ms，同分策略在本题没有体现时间收益。

5 题三：数值回传与剪枝顺序

Minimax 递归交替处理 MAX 和 MIN：叶子直接返回分值，MAX 取孩子的最大值，MIN 取最小值。Alpha-beta 在此基础上维护 alpha 和 beta；当 alpha 不小于 beta，当前分支已经无法改善祖先的选择，就停止访问剩余孩子。样例自底向上得到 MAX 层的 5、9、9、2，MIN 层的 5、2，最后根节点为 5。其中 alpha 初始为负无穷，记录 MAX 已知可保证的下界；beta 初始为正无穷，记录 MIN 已知可保证的上界。MAX 用较大值更新 alpha，MIN 用较小值更新 beta。

从左到右访问时，左侧 MIN 先拿到 5，于是下一个 MAX 的 beta 为 5。这个 MAX 读到叶子 6 后已有 alpha=6，满足截断条件，可以跳过后面的 9。完整 Minimax 访问 15 个节点、8 个叶子，Alpha-beta 访问 14 个节点、7 个叶子，答案均为 5。

我又反转了子节点顺序，发现样例仍然访问 14 个节点、7 个叶子。这个结果说明，换顺序并不必然改变剪枝数量。为进一步验证，我构造了 MAX(MIN(1,2,3), MIN(9,8,7))：正序访问 9 个节点，反序先取得 alpha=7，另一侧 MIN 读到 3 就截断，只访问 7 个节点，最终值都为 7。

这组对照更清楚地体现了排序的作用：先找到较强的候选，可以提前收紧搜索边界；但能省下多少搜索，取决于具体数值分布。

6 题四：宝藏分配的状态与距离

综合题中，我先把地图上的 A、B 和宝藏格统一转换为进入代价 1，障碍仍为 X，再用 A* 求六个位置之间的全部 36 个有向距离。这里终点收费，与题二不同，所有可通行格代价至少为 1，因此可以直接使用普通曼哈顿距离。结果见表 3，行表示出发点，列表示目的地。

表 3 A* 计算的最小移动代价矩阵

从 到	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

矩阵在本地图上恰好对称，但进入格子计费的一般加权图未必对称，所以代码没有只计算一半后直接复制。矩阵算好后，博弈阶段查表取得移动代价，避免在每个分支重复寻路。经过其他宝藏不自动收集，已获得的宝藏位置继续可通行，另一玩家也不会挡路。

状态保存双方位置 pos_a、pos_b，剩余宝藏 remaining，累计收益 score_a、score_b，以及当前玩家 player。每次选一个尚未取得且可达的宝藏，收益为宝藏价值减最小移动代价，然后更新行动方的位置、分数和剩余集合，最后切换玩家。A 最大化双方收益差 U，B 最小化 U。

这里不能省略位置：同一个宝藏，从初始位置和上一次取宝位置出发，代价可能不同。收益允许为负，轮到玩家仍需按题目规则取宝；不能因为“这一趟不划算”就自行跳过回合。到达深度 H 或宝藏取完时，直接用当前 score_a−score_b 评价局面，不额外猜测未来分数。

7 搜索深度为什么改变首步

表 4 各首步在最优对抗下的精确收益差

A 的首步	H 为 2	H 为 4
T1	−4	−5
T2	−1	1
T3	−2	3
T4	−4	−7

表 4 用完整枚举单独计算，避免把 Alpha-beta 内部截断返回的界误当成每个首步的精确价值。H=2 时 A 选 T2，收益为 6−4=2；B 随后选 T3，收益为 7−4=3，差值为 −1。H=4 时，T3 的完整价值达到 3，优于 T2 的 1，因此最优首步变为 T3。

H=2 只看双方各行动一次后的分数，对后续位置和剩余宝藏没有继续展开。H=4 则把全部四回合算完，能看到第一步移动对后面两次选择的影响。因此，浅层结果不能直接当成整个游戏的最佳决策。

8 完整博弈路线与节点排序改进

H=4 下的一条最优对抗路线是 A 取 T3，B 取 T1，A 再取 T2，B 最后取 T4。这里每一步都从该玩家上一次停留的位置出发，详细收益如下。

表 5 H 为 4 时的一条最优对抗路线

回合	玩家	出发点 到 宝藏	代价	本步收益	当前收益差
1	A	A → T3	8	-1	-1
2	B	B → T1	8	1	-2
3	A	T3 → T2	6	0	-2
4	B	T1 → T4	12	-5	3

最后 A 的累计收益为 −1，B 为 −4，所以 U=3。A 首次取 T3 本身会亏 1 分，但它改变了双方后续的位置和可选宝藏。把第一步的即时收益单独拿出来排序，会把 T2 排在前面；真正决定首步的仍应是后续搜索得到的完整效用。

实现基本剪枝后，我增加了一个简单的排序办法：每个玩家优先尝试自己“宝藏价值减移动代价”较大的目标。对 MIN 玩家，较高的自身收益会压低 A−B，因此同样按自己的即时收益降序排列。

表 6 题四搜索开销对照

深度	算法与访问顺序	访问节点	叶子	截断	中位耗时 ms
2	Minimax 自然顺序	17	12	0	0.0593
2	Alpha-beta 自然顺序	13	8	2	0.0437
2	Alpha-beta 反向顺序	16	11	1	0.0571
2	Alpha-beta 即时收益排序	11	6	3	0.0405
4	Minimax 自然顺序	65	24	0	0.2164
4	Alpha-beta 自然顺序	47	16	5	0.1682
4	Alpha-beta 反向顺序	46	16	4	0.1553
4	Alpha-beta 即时收益排序	37	12	6	0.1365

两种算法在所有这些配置下答案一致。H=4 时，默认剪枝把节点数从 65 降到 47，减少约 27.7%；即时收益排序后为 37，较完整搜索减少约 43.1%，较默认剪枝减少约 21.3%。中位耗时也从 0.2164 ms 降到 0.1682 ms，再降到 0.1365 ms。

顺序的影响并不固定。反向访问在 H=2 时比自然顺序多访问 3 个节点，在 H=4 时却少 1 个。即时收益排序在这两个深度都更好，但它只是启发式：若短期高收益导致后续位置不利，仍可能先展开较差分支；排序本身也会增加开销，所以不能保证任何树上都更快。

9 自设地图对照与改进效果

为了检验样例之外的表现，我设置了 20×20、40×40 的空地图、必须绕墙的地图，以及带随机权值和障碍的地图。随机图使用 101、202、303 三个种子，仅打通左侧和底边中原有的障碍来保证有路，保留随机地形代价。所有策略在同一场景上的最优代价一致。

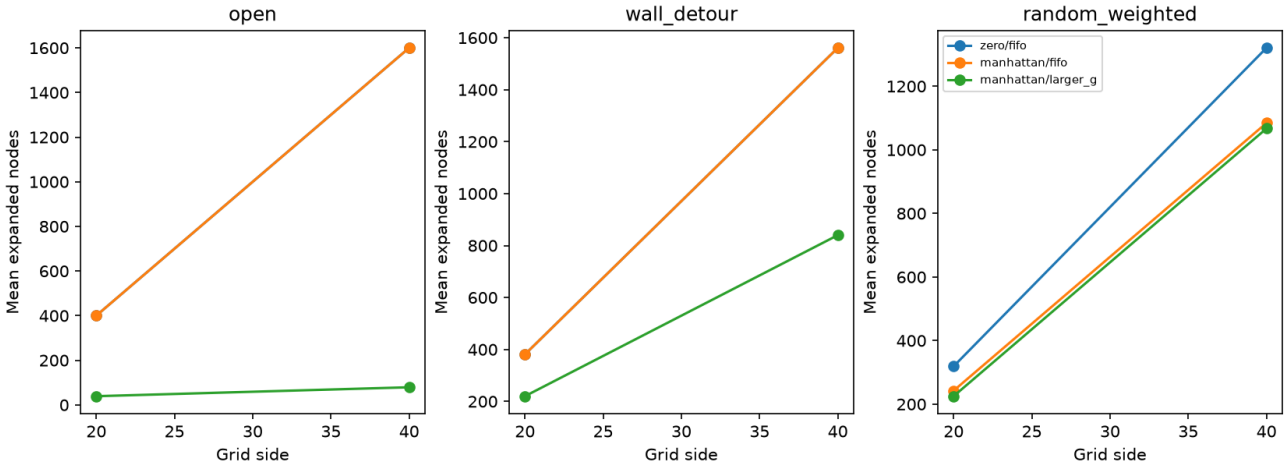


图 1 扩展地图的平均弹出节点数 随机加权图取三个种子的均值

空地图的结果最值得注意。40×40 时，h=0 和曼哈顿加 FIFO 都弹出 1600 个节点；相同 f 优先较大 g 后只弹出 79 个。原因是左上到右下的单位代价空图中，每个格子的 g+h 都为 78，FIFO 会在这个同分区域广泛展开。优先较大 g 能更快沿候选路径接近终点，因此即使 h 已经精确，同分规则仍很重要。

绕墙地图上，曼哈顿看不到必须向下再向上的绕行，40×40 时前两种策略都弹出 1561 个节点，较大 g 规则为 840 个。随机加权图中，这项改进没有空图上那么明显：例如种子 202 的 40×40 图，从普通曼哈顿的 1303 个降到 1295 个，耗时却从 2.8231 ms 增到 2.8986 ms，generated 也从 1440 增到 1489。少弹出几个节点，未必足以抵消排序和重复入堆的开销。

10 正确性核对与实验总结

除 7 项样例检查外，我用独立的重复松弛最短路程序核对了 960 次随机地图搜索，用完整递归核对了 400 次随机树搜索，并对 1200 次随机博弈配置检查最优值和首步。另核对 36 个有向距离项及 8 项边界情况，包括无解、起终点重合、阻塞、相邻免费终点和异常网格，均通过。随机测试能提供证据，但不能代替算法正确性的论证。

这次实验里，我最明显的体会是：实现规则要先于性能优化。题二的终点收费、题四的位置更新和负收益处理，都会直接影响答案；确定这些细节后，同分处理和即时收益排序才有比较的意义。两项改进都比较简单，但通过节点数和反例场景，能够看出它们什么时候有用、什么时候收益有限。

目前的扩展地图规模较小，随机地图也只有三个种子，耗时只代表本机测量。下一步若继续扩展，我会增加更复杂的地图和更多宝藏，再评估排序成本及缓存收益。